Islamic University of Technology (IUT)

Department of Software Engineering

Project Proposal

Real-Time CLI Chat Network

Team Members

Nafis Ahnaf Jamil — 230042150

Tabib Hassan — 230042131

Sieam Shahriare — 230042153

Supervisor:

Md. Tariquzzaman Faisal

Course:

SWE 4304 — Software Project Lab I

Date of Submission: December 14, 2025

Contents

1 Project Overview **2**

2 Motivation **2**

3 System Architecture **2**

3.1 High-Level Architecture 2

3.2 Server Architecture 2

3.3 Client Architecture 3

3.4 Communication Protocol 3

4 Key Features **4**

4.1 User Authentication 4

4.2 Direct Messaging 4

4.3 Group Chat 4

4.4 File Sharing 4

4.5 Message History 4

5 Technical Approach **5**

5.1 Programming Languages 5

5.2 Error Handling 5

6 Command Interface Design **5**

7 Timeline and Workflow **6**

7.1 Project Timeline 6

7.2 System UML Design 7

7.3 Server-Side UML Design 8

7.4 Development Workflow 9

8 Conclusion **9**

1. Project Overview

This project aims to develop a real-time command-line interface (CLI) based chat network using socket programming in C++. The system will allow multiple users to communicate simultaneously through direct messages and group conversations, share files, and maintain persistent message history—all without relying on any graphical user interface.

The application follows a client-server architecture where a central server manages all client connections, message routing, user authentication, and data persistence. The project emphasizes building core networking and system-level programming functionalities from scratch while adhering to the constraint of using only five library functions.

2. Motivation

Modern chat applications rely heavily on frameworks, libraries, and graphical components. This project removes these abstractions to help understand the fundamental principles of network communication, concurrent programming, and low-level system design.

3. System Architecture

3.1 High-Level Architecture

The system uses a centralized client-server architecture. The following diagram illustrates the overall structure:

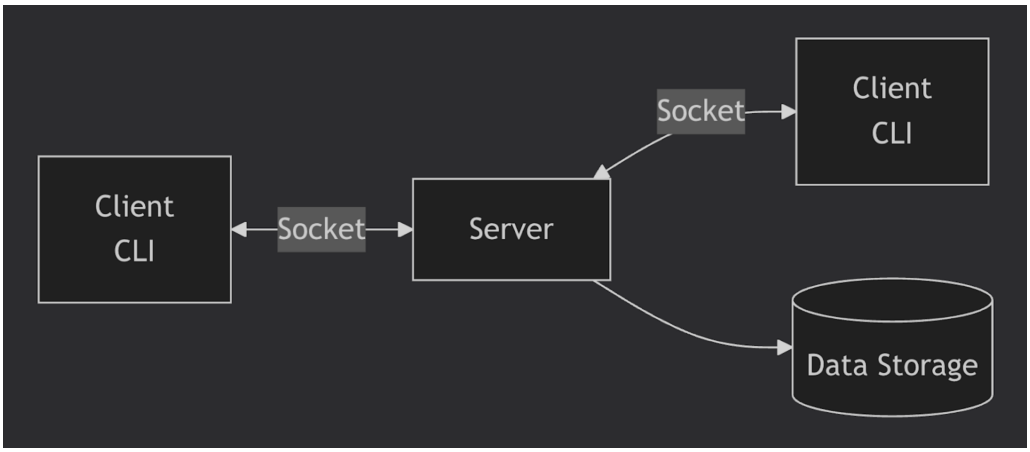

Figure 1: High-Level System Architecture

3.2 Server Architecture

The server is responsible for:

- **Connection Management:** Handles multiple clients concurrently using multithreading.
- **Message Routing:** For both direct and group communications.
- **User Management:** Authentication, session tracking, and maintaining user state.
- **Data Persistence:** Stores credentials, message history, and group information.
- **File Transfer:** Coordinates chunked binary file transmission.
- **Persistence Layer:** Reads and writes structured data to file-based storage.

3.3 Client Architecture

Each client implements:

- **CLI Interface:** Text-based command operations.
- **Socket Manager:** Maintains connection with server.
- **Input Handler:** Processes user commands.
- **Message Receiver:** Listens on a dedicated thread.
- **Display Manager:** Formats and prints messages.

3.4 Communication Protocol

The message packet includes:

- **Header:** Magic number for validation.
- **Type:** Identifier for message type.
- **Sender:** Username of sender.
- **Recipient:** Target user or group.
- **Length:** Payload size.
- **Timestamp:** Creation time.
- **Data:** Variable-length payload.

4. Key Features

4.1 User Authentication

- Signup with unique username and password.
- Login with credential verification.
- Active session management.
- Passwords stored using custom hashing.

4.2 Direct Messaging

- Real-time message delivery.
- Delivery acknowledgments.
- Offline message queueing.
- Chronologically threaded conversations.

4.3 Group Chat

- Group creation with unique names.
- Join and leave functionality.
- Broadcast messaging.
- Persistent group state across restarts.

4.4 File Sharing

- Binary file transfer (any file type).
- Chunk-based transmission.
- Metadata preservation.

4.5 Message History

- Retrieve past conversations.
- Persistent storage.
- Keyword and date-based search capability.

5. Technical Approach

5.1 Programming Languages

- **C++:** Server, client, sockets, concurrency, and file I/O.
- **SQL (conceptually):** Querying structured storage.

5.2 Error Handling

The system handles:

- Network failures.
- Invalid commands.
- File system errors.
- Concurrent access conflicts.
- Message delivery failures.

Recovery strategies include automatic reconnections and user-friendly error messages.

6. Command Interface Design

Commands (subject to change during development):

Authentication

- `/login <username> <password>`
- `/signup <username> <password>`
- `/logout`

Messaging

- `/dm <username> <message>`
- `/history <username> [limit]`

Group Commands

- `/group create <name>`

- `/group join <name>`
- `/group leave <name>`
- `/group msg <name> <message>`
- `/group members <name>`

File Commands

- `/file send <username> <filepath>`
- `/file list`

Utility

- `/list`
- `/status <message>`
- `/help`
- `/quit`

7. Timeline and Workflow

The project development will follow a structured and incremental workflow, ensuring steady progress and continuous validation of system components.

7.1 Project Timeline

- **Week 1:** Requirement analysis and protocol design.
- **Week 2:** System architecture and UML diagrams.
- **Week 3–4:** Server-side implementation.
- **Week 5:** Client-side CLI development.
- **Week 6:** File transfer and persistence.
- **Week 7:** Testing and documentation.

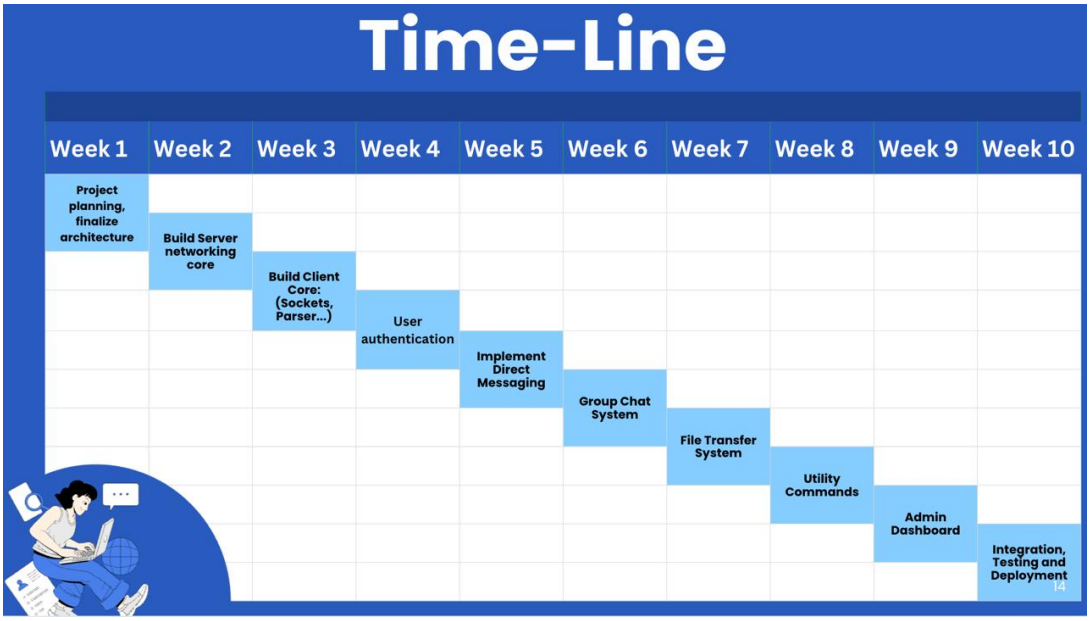

Figure 2: Project Work Timeline

7.2 System UML Design

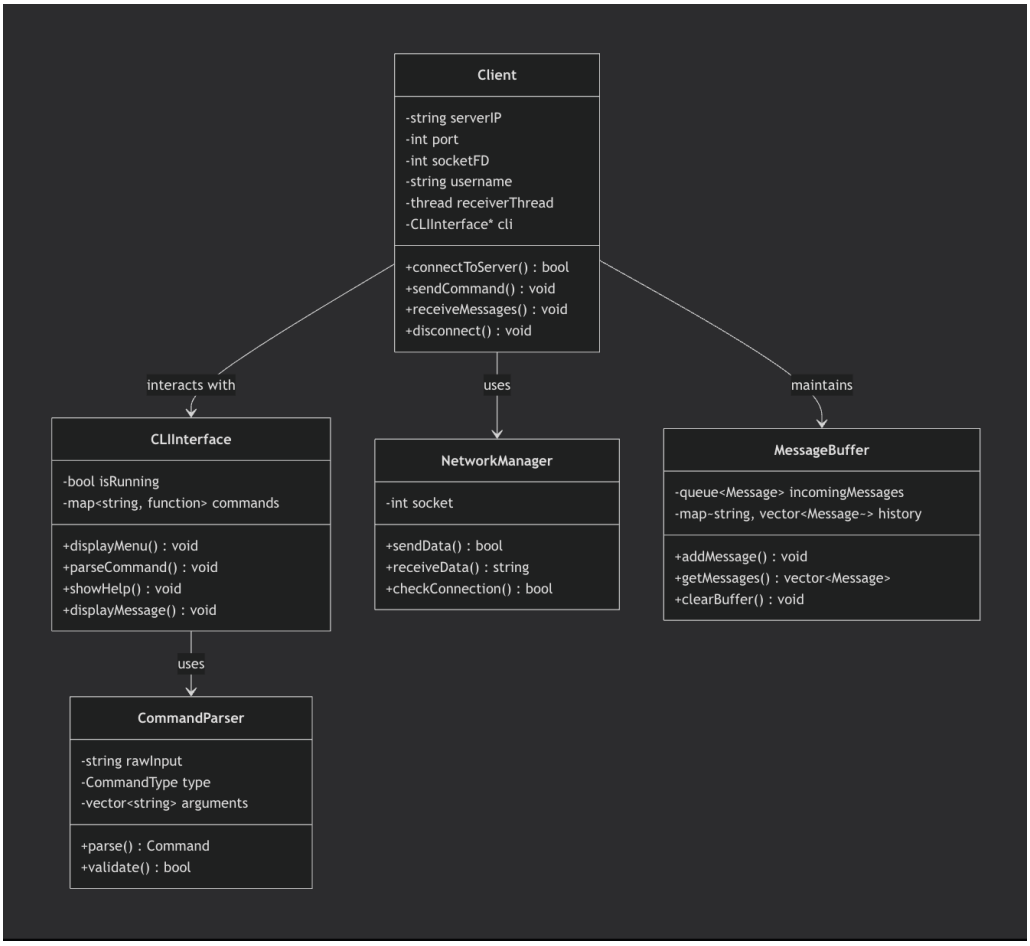

Figure 3: Overall System UML Diagram

7.3 Server-Side UML Design

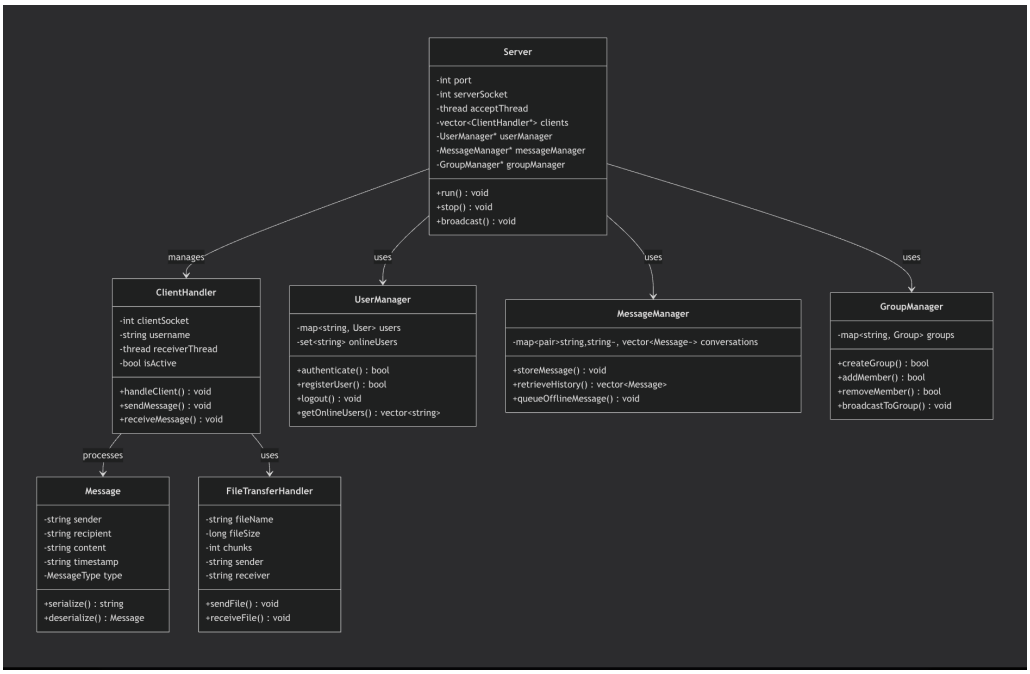

Figure 4: Server-Side UML Diagram

7.4 Development Workflow

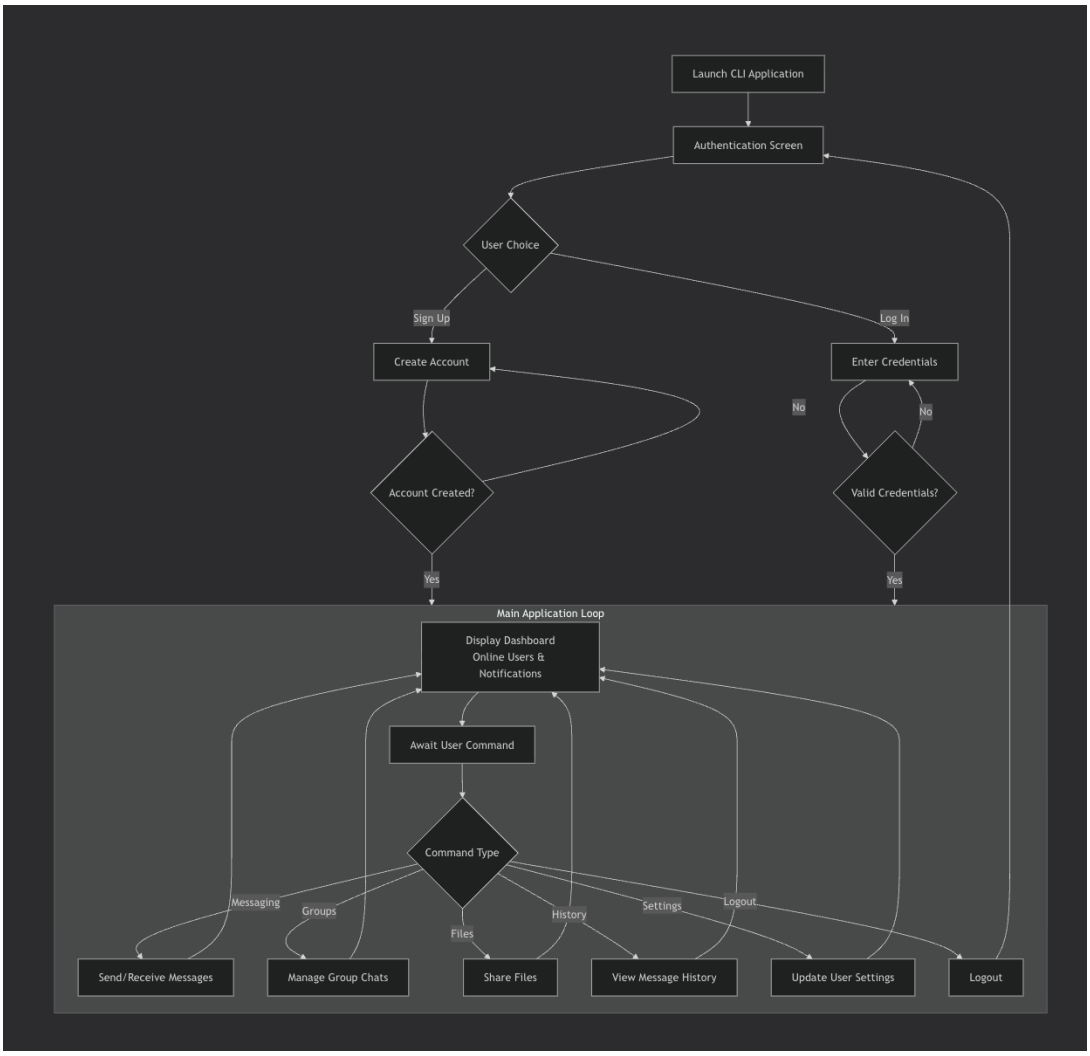

Figure 5: Client–Server Communication Workflow

8. Conclusion

This CLI-based chat network represents a challenging yet achievable SPL-I project. By implementing networking, concurrency, and persistence mechanisms from scratch with minimal library usage, the project emphasizes a strong foundational understanding of system-level software development.

The proposed architecture, workflow, and UML designs ensure modularity, scalability, and maintainability. Successful completion of this project will equip the team with practical experience relevant to advanced distributed systems and real-world software engineering practices.